

On-Premises Installation Runbook

What the office prepares, what the engineer does on site, and what comes back

Audience: the deployment engineer installing Vrodux ERP on a customer's own server, and the person in the office who prepares the handover pack.

Read this first: every step here works today, nightly cloud sync included. The install-time settings in §4.6 **seed** the schedule on first run; after that it is managed on screen at **Settings** → **Cloud Sync** — last run, run history, a manual **Sync now**, and a full re-send. Failure alerts go to whoever holds `settings.integrations.edit`, and to the addresses in `Sync:AlertEmails`.

0. The handover pack — what the office gives the engineer

One folder (USB stick or zip). Nothing in it should need to be typed twice on site.

Item	Where it comes from	Notes
Deploy/server/ folder, including output/	Run Deploy/server/publish.bat in the office	Self-contained — the server does not need .NET installed
VroduxERP-Setup-<version>.exe	npm run electron:build-win in FrontendVite/ → release/	The staff desktop client
Site sheet (below)	Filled in by the office	The only per-customer values
SQL Server Express installer	Microsoft, offline installer	Only if the server has no SQL Server
This runbook		

The site sheet — fill this in before the engineer leaves

```
Customer name      : _____
Tenant GUID       : _____ (from the cloud console)
Tenant slug       : _____
Device ID         : ____-____-____-____ (read ON SITE from the server — §1.2b)
License key       : _____ (long base64 string — paste, never retype)
License expires   : _____
Modules          : pos,inventory,finance (set on the tenant; the key carries them)
Plan              : _____

First admin email  : _____
First admin username : _____
First admin password : _____ (temporary — they change it on day one)

JWT secret        : _____ (generated fresh per site — see §1.3)

SMTP host / user / pass (optional, for invites + password resets):
_____
```

The license key is the important one. It is RSA-signed and carries the tenant GUID, slug, plan, seat limit and expiry, so the installation adopts the *existing* cloud tenant rather than creating a new workspace with a different id. Without it the box still runs, but it can never mirror to the cloud, because the two sides would not share a tenant id.

The key should also be bound to the customer's server (its Device ID — §1.2b). An unbound key runs on any computer it is copied to; a bound key runs only on the one machine it was issued for, checked entirely offline. That means the key is normally generated **during** the visit, not before it — the engineer reads the Device ID on site and sends it to the office.

1. In the office, before anyone travels

1.1 Create the cloud tenant (you have done this)

Super Admin → Tenants → Create: - Plan and modules as sold - **Deployment type:** OnPremises - **Is mirror:** on

Both sides are OnPremises. That is deliberate and not a typo — the licence generator refuses a Cloud tenant, and the mirror is a view of an on-premises installation licensed by the same key. See docs/on-premises-cloud-mirror.md §3.1.

1.2 Generate the license key

Set the tenant's modules **first** (Super Admin → that tenant → Modules), then → **Generate license** and choose validity days (e.g. 365). The module list is taken from the tenant itself — there is nothing to type, and nothing to get wrong. Copy the key and the tenant GUID onto the site sheet.

The key is the entitlement. The installation reads its modules and plan out of the signed payload, not out of `appsettings.json`. So **changing what a site runs means re-issuing the key**: change the modules in the cloud console, generate a new key, paste it into the box's `OnPremises:LicenseKey` and restart the service. The new plan and module list are picked up on that start. `OnPremises:Modules` is only a fallback for an old key issued without a module list.

1.2b Machine-bound keys — one key, one server (recommended)

A licence key can carry the **Device ID** of the server it is issued for. The installation compares it with its own ID on every start and every request — **offline, no internet needed** — and refuses to run anywhere else:

"This license key is registered to a different computer. This computer's code is 1A2B-3C4D-5E6F-7A8B — send it to Softaxis support for a key for this machine."

Leave the machine code blank and the key runs on any computer (every key issued before this feature is unbound and keeps working). Bind every new site.

The Device ID is derived from the Windows installation id of the server (the PC that runs the Vrodux service — not each till; every till shows the same ID because they all talk to one server). It survives reboots, reinstalling Vrodux, and IP or computer-name changes. It changes only if **Windows is reinstalled or the server is replaced** — then the customer needs a new key.

Reading it on site — any one of these; the first needs nothing at all:

Where	Needs
<code>Softaxis.ApiGateway.exe --device-id</code> in a command prompt, in <code>C:\Deploy\server\output\</code>	Nothing — no config, no database, no licence. Prints <code>Device ID: 1A2B-3C4D-5E6F-7A8B</code> and exits
The login page — under the sign-in form, with a Copy button	Service running with the <code>OnPremises</code> section filled in (licence not needed)
<code>http://localhost:5000/api/license/status</code> → <code>machineCode</code>	Same as above
Settings → Security → <i>Licence & Device</i> card	Licensed, signed in as admin/manager
Service startup log: <code>OnPremises: this computer's machine code is ...</code>	Service started with <code>OnPremises</code> filled in

The ID is **never shown on the cloud** (`erp.vrodux.com`): the server only publishes it when its own `appsettings.json` has `OnPremises:TenantName` OR `OnPremises:LicenseKey` set, which the cloud never does.

The workflow on the day: 1. Engineer copies the server build (\$4) and runs `Softaxis.ApiGateway.exe --device-id`. 2. Sends the Device ID to the office (WhatsApp / phone) and writes it on the site sheet. 3. Office: Super Admin → the tenant → **License Key** → paste it into **Machine code (optional)** → **Generate**. Send the key back. 4. Engineer pastes the key into `OnPremises:LicenseKey` (\$4.3) and continues the install as normal.

Record the Device ID in the site record (§13). It is not stored on the cloud tenant, so every renewal must be generated with the same machine code again — otherwise the renewal key is unbound.

1.3 Generate a JWT secret for this site

Every installation must have its **own** secret. The shipped `appsettings.json` contains the placeholder `__SET_JWT_SECRET_VIA_ENV_OR_DEV_SETTINGS__`, which is a known string — leaving it means anyone with the source can mint a valid token for that customer.

```
# 64 random bytes, base64 – run in the office, record on the site sheet
[Convert]::ToBase64String((1..64 | ForEach-Object { Get-Random -Maximum 256 })))
```

1.4 Build the two artefacts

Both from **PowerShell**, at the repo root. Absolute paths on purpose - see the warnings below.

```
# 1. Server - produces <repo>\Deploy\server\output\ (162 MB, self-contained)
& "D:\Shahbaz\Softaxis\ERP\Deploy\server\publish.bat"
```

```
# 2. Desktop client - produces FrontendVite\release\VroduxERP-Setup-<version>.exe (91 MB)
Set-Location "D:\Shahbaz\Softaxis\ERP\FrontendVite"
npm run electron:build-win
```

Build both from the same commit. The Activate-licence screen in the client calls an endpoint that only exists in the matching server build. Ship a new client against an old server and the button 404s - at exactly the moment a customer is trying to renew.

PowerShell will not run a script by bare relative path. `Deploy\server\publish.bat` is read as a *module* named `Deploy`, and the error says so: "The module 'Deploy' could not be loaded". Use `&` with the full path, or prefix `.\`. And `REM` is a `.bat` comment, not PowerShell - pasting it at a PS prompt is an error on its own line.

`publish.bat` **writes into the repo**, at `<repo>\Deploy\server\output\`. It does NOT update a deployed copy such as `C:\Deploy` - copy the result across yourself (§5), and back up the deployed `appsettings.json` first: a publish ships the repo's own version, with developer connection strings, a placeholder JWT secret and a blank `OnPremises` block.

2. Server prerequisites — check before starting

	Requirement	Why
OS	Windows Server 2019+ or Windows 10/11 Pro, x64	The service is published <code>win-x64</code>
SQL Server	2019+ — Express is fine	Change Tracking (needed later for cloud sync) works on Express; CDC does not
Disk	20 GB free minimum	Database, logs, backups
Rights	Local administrator	Installing a service, firewall rule
SQL rights	The service account needs db_owner on the database	Migrations run at startup, and Change Tracking needs it
Network	Static LAN IP, and the port open to staff PCs	Tills and desktops connect to it
Power	UPS strongly recommended	It is the shop's system of record

The server must have a fixed IP. Every desktop client is pointed at it by address; a DHCP lease change means every PC stops working at once.

3. SQL Server and the database

1. Install SQL Server Express if absent. During setup enable **Mixed Mode** if you intend to use a SQL login, and note the instance name (`SQLEXPRESS` by default).
2. Create an empty database: `sql CREATE DATABASE SoftaxisErpDb`; **Empty is correct — do not restore anything into it.** The service creates every table itself on first start, because an installation carrying an `OnPremises:LicenseKey` applies its migrations automatically. (A box with no licence key configured will **not** create tables — another reason §4.3 is not optional.)
3. **Grant the service account access to the database. This is not optional.**

`install-service.bat` runs the service as `LocalSystem`, and the connection strings use `Integrated Security=true` — so the database is opened as `NT AUTHORITY\SYSTEM`, not as you. That account has a login on the instance by default but **no user in your database**, so without this the service cannot read a single table.

Run it, substituting your instance and database name:

```
powershell sqlcmd -S "<SERVER>\SQLEXPRESS" -E -C -d <DATABASE> -Q "CREATE USER [NT AUTHORITY\SYSTEM] FOR LOGIN [NT AUTHORITY\SYSTEM]; ALTER ROLE db_owner ADD MEMBER [NT AUTHORITY\SYSTEM];"
```

`db_owner` is required, not generous: the service creates and alters tables on every upgrade, which `db_datareader` / `db_datawriter` cannot do.

Verify — this must return **1**: `powershell sqlcmd -S "<SERVER>\SQLEXPRESS" -E -C -d <DATABASE> -Q "SELECT COUNT(*) FROM sys.database_principals WHERE name='NT AUTHORITY\SYSTEM'"`

This is the classic 1053. Skip it and the exe runs perfectly from a console — because a console runs as *you*, and you own the database — while the service dies instantly with "The service did not respond to the start or control request in a timely fashion" and `WIN32_EXIT_CODE : 0`. Same binary, same config, different account.

Using a SQL login instead? Create one, grant it `db_owner`, put it in the connection strings, and skip the grant above — the service then authenticates as that login rather than as the machine account. 4. Confirm you can connect with whichever account you chose before going further. Every later failure looks the same from the outside; proving the database now saves an hour.

4. Copy and configure the server

1. Copy the `Deploy\server\` folder to the machine — e.g. `C:\Vrodux\server\`.
2. Open `C:\Vrodux\server\output\appsettings.json` in a text editor.

4.1 Connection strings — all of them

There are **18** connection-string entries — `IdentityDb`, `POSDb`, `InventoryDb`, `SalesDb`, `PurchaseDb`, `HrDb`, `FinanceDb`, `CrmdB`, `ConstructionDb`, `RealEstateDb`, `HospitalityDb`, `RestaurantDb`, `RecipeDb`, `ProjectManagementDb`, `VisaDb`, `SupportDb`, `AiAssistantDb`, `NotificationsDb`.

They all point at the same physical database — one database, a schema per module. Do a find-and-replace of the server portion rather than editing eighteen lines by hand:

```
Server=<SERVERNAME>\SQLEXPRESS;Database=SoftaxisErpDb;Integrated Security=true;MultipleActiveResultSets=true;TrustServerCertificate=True;
```

Missing one is a real failure mode: the gateway starts, most of the product works, and the one module whose string still points at the office machine fails at startup and takes the whole service down with it. Search the file for the old server name afterwards and confirm zero hits.

4.2 The JWT secret

```
"Jwt": {  
  "Secret": "<the value from the site sheet>",  
  ...  
}
```

⚠ Changing this later invalidates every stored 2FA secret and every refresh token — users must re-enrol their authenticator. Set it once, now, and keep it on the site sheet.

4.3 Adopt the cloud tenant

```
"OnPremises": {
  "LicenseKey": "<the long key from the site sheet>",
  "TenantName": "Customer Name",
  "Modules": "pos,inventory,finance",
  "ContactEmail": "owner@customer.com",
  "Country": "United Arab Emirates",
  "AdminEmail": "owner@customer.com",
  "AdminUsername": "owner",
  "AdminFirstName": "First",
  "AdminLastName": "Last",
  "AdminPassword": "<temporary password from the site sheet>"
}
```

What this does on first start: validates the key's signature, creates the tenant **with the GUID the key carries**, provisions its roles, and creates the first administrator, pre-verified so they can sign in immediately (there is no mailbox on a shop counter).

- Leave `LicenseKey` blank and nothing is adopted — you get an unlicensed box that blocks every request. Do not skip it. (Read the Device ID with `--device-id` **before** the first start, so the bound key is already in hand when you get here — §1.2b.)
- A key bound to a different machine is refused at this step with the machine's own code in the log — generate a new key for that code.
- If the key is invalid or expired the log says so and **no tenant is created** — it never invents one from an unverified key.
- Re-running is safe: it only re-asserts deployment type, license and mirror flag.

Do not set the `SuperAdmin` section on a customer's server. That is a platform-level account for your own cloud; the customer's administrator comes from `OnPremises:Admin*` above.

4.4 Email (optional but recommended)

```
"Email": {
  "SmtpHost": "...", "SmtpPort": "587",
  "SmtpUsername": "...", "SmtpPassword": "...",
  "FromAddress": "...", "FromName": "Customer Name"
}
```

Without it, password-reset and invite emails cannot be sent. Everything else works; account hand-over falls back to temporary passwords shown on screen.

4.5 FrontendUrl — read the caveat

`FrontendUrl` is the address password-reset and invite links point to. **If the site uses only the desktop client, there is no web address to point at** — the gateway does not serve the web UI, and the client loads its own bundled copy. Those emailed links will not open anywhere useful.

Practical consequence for a desktop-only site: reset passwords through **Settings → Users → Reset password** (which shows a temporary password on screen) rather than the "forgot password" email. The

service logs a warning at startup if `FrontendUrl` is unset or points at localhost — that warning is expected on such a site, not a fault.

4.6 Nightly cloud sync

```
"OnPremises": {  
  // ... the keys from 4.3, plus:  
  "SyncCloudUrl":      "https://erp.vrodux.com",  
  "SyncRunAtLocalTime": "23:30",  
  "SyncTimeZoneId":     "Asia/Dubai",  
  "SyncEnabled":        "true"  
}
```

- **Leave `SyncCloudUrl` blank** for a site that does not mirror. Everything else still works.
- **The time is local wall-clock, in `SyncTimeZoneId`** — not UTC. Pick a time after close but before anyone switches the machine off. If the box is off at that moment the run still happens, later the same day.
- **These seed the schedule once.** After the first start the settings live in the database, and a restart never overwrites them — so a later change must go through the API (§9), not this file.

5. Install and start the service

⚠ Build the schema from a console FIRST — before installing the service

The first start creates several hundred tables and seeds roles and permissions. That takes minutes, and **the Service Control Manager only waits 30 seconds** — it kills the process and reports 1053, then EF rolls the partial migration back, so the database stays empty however many times you retry.

Run it once in a console, where nothing is timing it:

```
C:\Vrodux\server\output\Softaxis.ApiGateway.exe
```

Wait for these two lines:

```
Licensing: installing schemas for N module(s) - ...  
OnPremises: adopted workspace ... from the license key.
```

Confirm the schema exists — expect a few hundred, not 0:

```
sqlcmd -S "<SERVER>\SQLEXPRESS" -E -C -d <DATABASE> -Q "SELECT COUNT(*) FROM sys.tables"
```

Then `Ctrl+C` and install the service. That start has nothing left to migrate and comes up in seconds.

Now install the service

Right-click → **Run as administrator:**

```
C:\Vrodux\server\install-service.bat
```

It creates the `VroduxERP` service (auto-start, `LocalSystem`), sets it to restart on failure (5 s / 10 s / 30 s), and starts it.

Confirm it is actually up

```
sc query VroduxERP
curl http://localhost:5000/health
```

`/health` must return **200** and list the services. If the service starts and immediately stops, open **Event Viewer** → **Windows Logs** → **Application** — a bad connection string and a missing `db_owner` both land there with a clear message.

6. Firewall

Only if staff connect from other PCs (they almost always do):

```
C:\Vrodux\server\open-firewall.bat (as administrator)
```

Opens **TCP 5000** on the **domain and private** profiles only — deliberately not public. If the customer's network profile is set to Public, either change the profile or add the rule manually; otherwise the port is open on the server and unreachable from every till.

Verify from a staff PC, not from the server:

```
curl http://<SERVER-IP>:5000/health
```

7. First sign-in and hand-over to the customer

From a desktop client or browser on the LAN, sign in as the `OnPremises:AdminEmail` account with the temporary password.

Immediately, with the owner present: 1. **Change the administrator password**. 2. Settings → General: company name, legal name, address, tax registration number, logo, currency. The tax number and logo appear on invoices and quotations — set them before the first document. 3. Settings → Users: create their real staff accounts and assign roles. 4. Settings → Security: agree a password policy and lockout threshold if they want one.

Hand over the site sheet to the customer's owner, or destroy the copy of the admin password on it. It should not stay in a shared folder.

8. Install the desktop client on staff PCs

On each till and office PC:

1. Run `VroduxERP-Setup-<version>.exe`. It installs per-machine with desktop and Start-menu shortcuts.

2. Launch **Vrodux ERP**. It defaults to `http://localhost:5000`, which is only correct on the server itself.
3. Point it at the server: **tray icon** → **Server Settings** → **Change** → `http://<SERVER-IP>:5000`, then it reloads. The address is saved per machine (`vrodux-config.json` in the user's app data), so it survives restarts and upgrades.
4. Sign in and confirm the modules the customer bought are present.

Do this on **one** PC and confirm end to end before rolling out to the rest. A wrong IP, a closed firewall and a stopped service all look identical from the client.

9. Nightly cloud sync

This pushes the day's data to the cloud mirror created in step 1. It works; it is configured in §4.6 before the service first starts.

What it does

At the configured local time the installation sends everything that changed since the last successful run — POS, Inventory, Finance and the identity records that make the mirror readable — to the cloud workspace with the same tenant id. The first run is a full export; every run after it sends only changes. The cloud copy is **read-only**: it accepts no edits and runs no background jobs, so nothing is generated twice.

Confirm it on site, before you leave

The schedule is at 23:30 by default, so do not wait for it. Sign in as the administrator and run it by hand:

```
# from the server, after signing in and getting a token
curl -X POST -H "Authorization: Bearer <token>" http://localhost:5000/api/sync/run
```

A good first result looks like this — the number of rows depends on how much data exists:

```
{ "ran": true, "succeeded": true, "tablesProcessed": 73, "rowsSent": 1564, "tablesFailed": 0 }
```

Then check the current state:

```
curl -H "Authorization: Bearer <token>" http://localhost:5000/api/sync/settings
```

`lastSuccessAt` should be a moment ago, `consecutiveFailures` 0, `licenceConfigured` true.

If it does not run

skippedReason or error	Means
"no cloud-sync configuration"	§4.6 was left blank — set <code>SyncCloudUrl</code> and restart
"Cloud sync is switched off"	<code>SyncEnabled</code> is false
"No licence key is configured"	§4.3 — the push authenticates with the licence
"the mirror refused ... 401"	The cloud tenant does not match this licence
"that workspace is not configured as a cloud mirror"	Is mirror was not ticked when the cloud tenant was created
"Could not reach the cloud mirror"	Outbound HTTPS is blocked, or the URL is wrong

Changing the time later

Open **Settings** → **Cloud Sync** in the app and change it there — it takes effect on the next night with no restart. The same screen shows whether the last run worked, the recent run history, and a **Sync now** button for checking a fix without waiting until 23:30.

If the app is not reachable (mid-install, or you are on the server console), the same thing over the API:

```
curl -X PUT -H "Authorization: Bearer <token>" -H "Content-Type: application/json" -d '{"enable":true,"cloudBaseUrl":"https://erp.vrodux.com","runAtLocalTime":"23:30","timeZoneId":"Asia/Dubai"}' http://localhost:5000/api/sync/settings
```

Note the ordering: install-time settings only **seed** the schedule the first time. Once a row exists, a restart never overwrites it — so a change made on the screen (or over the API) sticks, and editing `appsettings.json` afterwards will appear to do nothing.

If the mirror is wrong rather than merely behind — after restoring the cloud database, say — use **Re-send everything** on that screen. It clears every table's position so the next run exports the whole database again. It deletes nothing on either side (the receiver upserts), but on a busy site that run takes a long time.

Alerting

Set `Sync:AlertEmails` in `appsettings.json` to the office addresses that should hear about a broken push — usually your own support desk rather than the shop's staff, since the person who can re-key a licence or open a firewall is rarely on site. Administrators at the shop also get an in-app notification linking straight to the screen.

Alerts are sent **twice per outage, not once per attempt**: on the first failure, and again once it has failed three times. A channel that repeats itself nightly is one people filter out, exactly when it next matters.

What it is not

The mirror is not a backup. It is read-only and holds only what was pushed. §10 still applies.

9b. Trials, renewals and upgrades — the licence is the subscription

An on-premises installation has no subscription. It is gated entirely by the **RSA-signed licence key**, re-checked on every request, fully offline. The expiry is inside the signed payload, so it cannot be extended by editing the database or `appsettings.json` on site.

Selling a 30-day trial

Generate the key with **validity 30 days**. That is the whole mechanism — no separate trial flag. On day 31 every request is refused with `LICENSE_EXPIRED` and the shop stops trading.

Tell the customer the date. The app shows a countdown banner from 30 days out, turning red inside the last week, but do not rely on someone noticing it. A till that stops mid-morning is a phone call you do not want.

When they pay — issuing the new key

1. Super Admin → the tenant → set its **modules** to what they bought.
2. **Generate license** with the real validity (e.g. 365 days) and the **same machine code** as before (from the site record — or ask the customer: it is on their login page and in Settings → Security). Leaving it blank issues an unbound key.
3. Send them the key.

The key carries the plan and the module list, so this is also how you add or remove a module after go-live. Changing modules in the cloud console alone does nothing to their server.

Installing it on their server — no visit, no restart

The customer pastes the key themselves:

- **If the licence has already lapsed**, the blocked screen shows an **Activate a licence key** box. `/api/license/` is exempt from the enforcement middleware for exactly this reason, so it works when everything else is refused.
- **Before it lapses**, the same thing is reachable at `/subscription-expired`.

It takes effect immediately — the plan, the module list and the expiry are all re-asserted from the signed payload, the service is **not** restarted, and nothing else on the installation is touched. Have them sign out and back in afterwards so their token picks up any new modules.

Editing `OnPremises:LicenseKey` in `appsettings.json` and restarting the service still works and is equivalent. Use it only when nobody can reach the UI at all.

10. Backups — do not skip this

The mirror, when it exists, is **not a backup**: it is read-only and holds only what was pushed. This server is the customer's system of record.

Set up a SQL Server backup job before leaving: - Full backup nightly, to a **different physical disk** than the database. - Weekly copy off-site (external drive rotated, or cloud storage). - **Test a restore once**, on the day, into a scratch database. An untested backup is a belief.

`docs/backup-restore-guide.md` has the commands.

11. Leaving checklist

Nothing here is optional.

- ☐ `sc query VroduxERP` → RUNNING, start type Automatic
- ☐ `/health` returns 200 **from another PC on the network**
- ☐ Service survives a reboot — actually reboot the server and re-check
- ☐ Administrator password changed by the owner
- ☐ Company details, tax number and currency set
- ☐ At least one staff account created and signed in
- ☐ Desktop client working on every PC, each pointing at the server IP
- ☐ **Cloud sync run by hand and confirmed** — `succeeded: true`, `tablesFailed: 0`
- ☐ Backup job created **and a restore tested**
- ☐ JWT secret, license key and tenant GUID recorded in the office record for this site
- ☐ Admin password removed from any shared copy of the site sheet

Report back to the office

```
Site           : _____
Installed on   : _____
Server name/IP : _____
SQL instance   : _____
Tenant GUID    : _____ (confirm it matches the cloud console)
License expires : _____
Backup location : _____
Sync run time  : _____ (confirmed working on site: yes / no)
```

12. Troubleshooting

Symptom	Likely cause	Check
<code>StartService FAILED 1053</code>	<code>NT AUTHORITY\SYSTEM</code> has no user in the database — the most common cause by far	§3.3 — run the grant. The exe works from a console (that runs as <i>you</i>) while the service dies: same binary, different account
1053 on a brand-new database	The first migration outran the SCM's 30-second limit	§5 — run the exe from a console once to build the schema, then install the service
Service starts then stops	Bad connection string, or no <code>db_owner</code>	Event Viewer → Application
Service runs but the database has no tables	No <code>OnPremises:LicenseKey</code> , so migrations were skipped	§4.3 — set the key and restart, or set <code>Database:MigrateOnStartup: true</code>
A licensed module is missing from the sidebar	The licence was issued before that module was added to the tenant	Add it in the cloud console, regenerate the key, paste and restart — the signed key decides, not <code>OnPremises:Modules</code>
A module the customer never bought is visible	Old build	Fixed — the schema is no longer even created for unlicensed modules
<code>/health</code> works locally, not from a PC	Firewall, or Public network profile	<code>open-firewall.bat</code> ; check the profile
"No license key has been issued" on every request	<code>OnPremises:LicenseKey</code> blank or wrong	The key must be pasted whole
"Your software license has expired"	Key past its expiry	Generate a new one in the cloud console
"This license key is registered to a different computer" (<code>LICENSE_WRONG_MACHINE</code>)	Key was bound to another server, or Windows was reinstalled / the server replaced	The message shows this machine's Device ID — generate a new key with it
Device ID shows <code>UNKNOWN</code> / is missing	Windows installation id could not be read (very rare)	Issue an unbound key for this site and investigate
Licence generation in Super Admin fails: "License signing is not configured"	Cloud server has no signing key	Set <code>LICENSE_RSA_PRIVATE_KEY_PEM</code> in <code>/opt/vrodux/shared/.env</code> and recreate the api container (§14)
Login says the workspace is unavailable	Tenant adopted with the wrong id, or deleted in the cloud	Compare the tenant GUID against the console
One module 500s, the rest work	A connection string still points at the office machine	Search <code>appsettings.json</code> for the old server name
Password-reset emails go nowhere	No SMTP, or <code>FrontendUrl</code> unreachable	§4.4 and §4.5 — use admin reset instead
Everything worked, now nothing does after an upgrade	New <code>appsettings.json</code> overwrote the site's	Back up <code>appsettings.json</code> before every upgrade

Upgrading a site later

1. **Copy `appsettings.json` somewhere safe.**
2. Stop the service: `sc stop VroduxERP` .

3. Replace `output\` with the new build.
4. Restore the saved `appsettings.json` — or re-apply §4 to the new one if its structure changed.
5. Start: `sc start VroduxERP`. Migrations apply automatically.
6. Re-check `/health` and one real screen.

13. What the office keeps, per site

One record per customer, treated as credentials — not a shared spreadsheet: tenant GUID, slug, **Device ID**, license key + expiry, JWT secret, server name/IP, SQL instance, install date, build version, backup arrangement, and (when it ships) the sync time and cloud URL.

You will need the tenant GUID and license key again for every upgrade, every license renewal, and the first time cloud sync is switched on.

14. Licence signing — cloud only

Licence keys are RSA-signed with a key pair:

	Private key	Public key
Does	Creates licence keys	Verifies licence keys
Lives	Only on the cloud server , <code>/opt/vrodux/shared/.env</code> → <code>LICENSE_RSA_PRIVATE_KEY_PEM</code>	Compiled into the software (<code>LicenseService.cs</code> → <code>EmbeddedPublicKeyPem</code>) — every install has it
Set up	Once, ever	Ships with every build

Never put the private key on a customer's server. Anyone holding it can mint a licence for any customer, any expiry, any modules. Customers only ever receive licence keys.

The key pair in use

The original pair was embedded in the exe (private half included) and is in git history, so it was replaced. The current pair was generated on a development machine with OpenSSL (RSA-2048); the private half was uploaded to the server and the public half compiled in. **Every licence signed with the old pair is rejected by builds carrying the new public key** — each site needs the new build **and** a new key, installed together.

Setting it up / rotating it again

1. Generate (PowerShell — OpenSSL ships with Git for Windows): `& "C:\Program Files\Git\usr\bin\openssl.exe" genrsa -out license-private.pem 2048 & "C:\Program Files\Git\usr\bin\openssl.exe" rsa -in license-private.pem -pubout -out license-public.pem`
2. **Back up license-private.pem offline** (password manager / encrypted USB). Lose it and no licence can be issued or renewed until you rotate and re-key every site.
3. Upload and register it on the server: `scp license-private.pem root@<server>:/opt/vrodux/shared/`
`chmod 600 /opt/vrodux/shared/license-private.pem` `echo "LICENSE_RSA_PRIVATE_KEY_PEM=$(awk 'NF`

```
{printf "%s\\n", $0}' /opt/vrodux/shared/license-private.pem | sed 's/\\n$//')" >>
```

```
/opt/vrodux/shared/.env
```

4. Put `license-public.pem` into `EmbeddedPublicKeyPem` in `LicenseService.cs` — copied exactly; base64 must contain **no spaces** (editors like to turn a `+` into `+`).
5. Deploy, then load the env into the API (a plain `docker restart` does **not** re-read `.env`): `cd /opt/vrodux/current && docker compose -f docker-compose.prod.yml --env-file /opt/vrodux/shared/.env up -d --no-deps --force-recreate vrodux-api`
6. Verify without printing the secret: `docker exec vrodux-api sh -c 'v="$License__RsaPrivateKeyPem"; [-n "$v" ] && echo "set, length ${#v}" || echo MISSING'` Then Super Admin → Generate license. "License signing is not configured on this server" means the variable did not reach the container.
7. Delete the local copy of the private key.
8. Re-issue every site's key (bound to its Device ID) and install it with the new build.